



南開大學
Nankai University

数据结构实验报告一

递归

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网路空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.4 代码	2
1.5 测试用例截图	3
2 题目二	4
2.1 题目表述	4
2.2 代码 2 的测试用例	4
2.3 思路	4
2.4 代码	4
2.5 测试用例截图	6

1 题目一

1.1 题目表述

编写递归函数计算 Fibonacci 数列，能避免重复计算。

输入：input.txt，仅包含一个整数 n ($0 - 90$)

输出：程序应能检查输入合法性，若有错误，输出错误提示“WRONG”；否则输出 $F(n)$ 。两种情况都输出一个回车（形成一个空行）。所有实例均应在 30 秒内输出结果。

提示：可用一数组保存 Fibonacci 数列，用一个特殊值表示还未计算出 Fibonacci 数，递归调用前先检查数组，若已计算，直接取用，不进行递归调用；若未计算，调用递归函数，计算完成后保存入数组。实际上，这种方法得到了 $F(1) - F(n)$ ，而不仅是 $F(n)$ 。

另外，注意数据类型取值范围问题。VC 6.0 中，长整型为 LONGLONG，而其输出用 %I64d。

1.2 代码 1 的测试用例

5、-5

1.3 思路

为了确保“所有实例均应在 30 秒内输出结果”，引入一个记忆数组 memo 将已计算的结果缓存起来，避免重复工作（若 $\text{memo}[n]$ 已被计算，则直接返回 $\text{memo}[n]$ ）。

使用递归 $\text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$ ，计算 Fibonacci 数列。

1.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 using namespace std;
5 vector<long long> memo;
6 long long fib(int n) {
7     if (n == 0) return 0;
8     if (n == 1) return 1;
9     if (memo[n] != -1) {
10         return memo[n];
11     }
12     memo[n] = fib(n - 1) + fib(n - 2);
13     return memo[n];
14 }
15 int main() {
16     ifstream fin("input.txt");
17     if (!fin.is_open()) {
18         cout << "WRONG" << endl << endl;
19         return 0;
```

```
20     }
21     int n;
22     if (!(fin >> n)) {
23         cout << "WRONG" << endl << endl;
24         fin.close();
25         return 0;
26     }
27     fin.close();
28     if (n < 0 || n > 90) {
29         cout << "WRONG" << endl << endl;
30         return 0;
31     }
32     memo.assign(n + 1, -1);
33     if (n >= 0) memo[0] = 0;
34     if (n >= 1) memo[1] = 1;
35     long long result = fib(n);
36     cout << result << endl << endl;
37     return 0;
38 }
```

1.5 测试用例截图



图 1.1: 测试用例 5 的输出结果

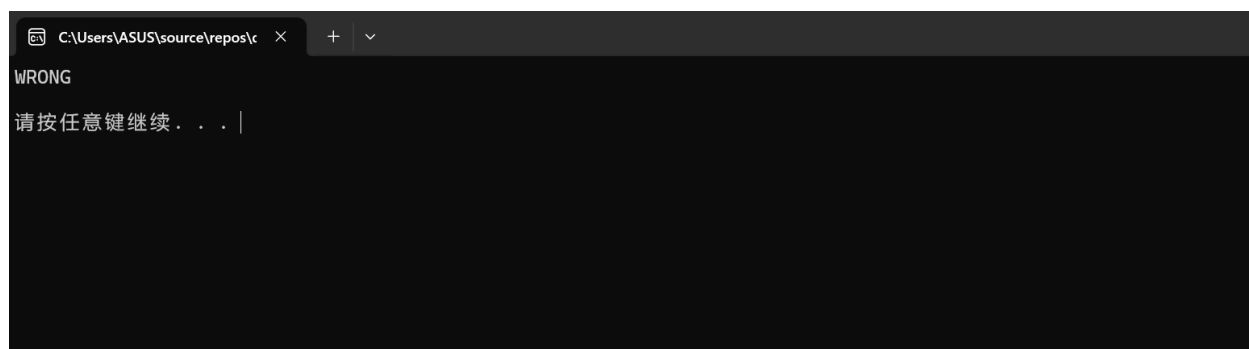


图 1.2: 测试用例-5 的输出结果

2 题目二

2.1 题目表述

编写递归函数求 n 个元素集合的所有子集。不妨令集合元素为小写字母,原集合为 $\{ 'a', 'b', \dots, 'a' + n - 1 \}$ 。

输入: input.txt, 仅包含整数 n ($1 \leq n \leq 26$)。

输出: 若输入合法, 输出集合的所有子集; 否则输出 “WRONG”。子集输出格式为每行一个子集, 空集用空行表示, 非空集合每个元素间用一个空格间隔, 最后一个元素之后不能有空格。例如, 对 $n=3$, 可能的输出为:

```
a
a b
a b c
a c
b
b c
c
```

提示: 递归函数一般设计思路是将问题求解分为若干阶段, 每个阶段有多种选择, 则每个阶段尝试各种选择就构成函数主体, 而“进入下一阶段”则用递归调用完成。对本问题, 子集中包含元素的选择可作为“阶段”, 而每个元素“是否在子集中”可作为“每个阶段的多种选择”。有个地方需要一些技巧: 在何时安排输出, 可达到要求的顺序? 常用技巧: 集合用什么样的数据结构保存、处理一般高级程序设计语言都不提供“集合”数据类型, 因为集合大小不固定, 且进行运算大小变化可能很剧烈。一般思路是用数组或链表(线性表)来保存集合, 数组元素(链表节点)对应集合元素, 集合运算的效率较差, 可能会频繁改变数组、链表大小。但如果可能的集合元素是有限个, 如本题, 有一种较好的方法, 仍然可用数组, 但数组元素不是保存集合元素, 而是每个数组元素固定对应一个可能的集合元素, 数组元素类型是布尔值 (t/f, 0/1), 表示对应元素是否包含在本集合内, 显然本题用此方法很适合。还可进一步优化, 由于每个数组元素值只可能是 0/1, 因此无需占用一个整型字, 占用一位即可, 这种“位表示法”的另一好处是将集合的操作转换为二进制位运算, 效率非常高。比如, 对本题, 1101 可能就表示 a, c, d (a - 最低位), 那么 $\{a, d, e, f\}$ 的位表示是什么? 两个集合的并集用两个二进制数的什么运算即可完成? 交集呢? 如果使用这种方法, 无需递归即可得到所有子集, 当然, 按题目要求, 大家还应该设计递归函数。

2.2 代码 2 的测试用例

5、-5

2.3 思路

以 $a[27]$ 和 len 为核心状态变量, 通过递归函数 $generate(cur, n)$ 对每个元素做出“包含”或者“不包含”的判断。 $a[]$ 作为路径数组记录当前子集的元素索引, len 作为长度指针控制输出范围。当所有元素决策完毕 ($cur == n$) 时, 程序遍历 $a[0..len - 1]$, 将索引转换为字母并按格式输出。

2.4 代码

```
1 #include <iostream>
2 #include <fstream>
3 using namespace std;
4 char a[27];
5 int len = 0;
6 void generate(int cur, int n) {
7     if (cur == n) {
8         for (int i = 0; i < len; i++) {
9             if (i > 0) cout << " ";
10            cout << (char)('a' + a[i]);
11        }
12        cout << endl;
13        return;
14    }
15    generate(cur + 1, n);
16    a[len] = cur;
17    len++;
18    generate(cur + 1, n);
19    len--;
20 }
21 int main() {
22     ifstream fin("input.txt");
23     if (!fin.is_open()) {
24         cout << "WRONG" << endl;
25         system("pause");
26         return 0;
27     }
28     int n;
29     if (!(fin >> n)) {
30         cout << "WRONG" << endl;
31         fin.close();
32         system("pause");
33         return 0;
34     }
35     fin.close();
36     if (n < 1 || n > 26) {
37         cout << "WRONG" << endl;
38         system("pause");
39         return 0;
40     }
41     len = 0;
```

```
42     generate(0, n);  
43     system("pause");  
44     return 0;  
45 }
```

2.5 测试用例截图

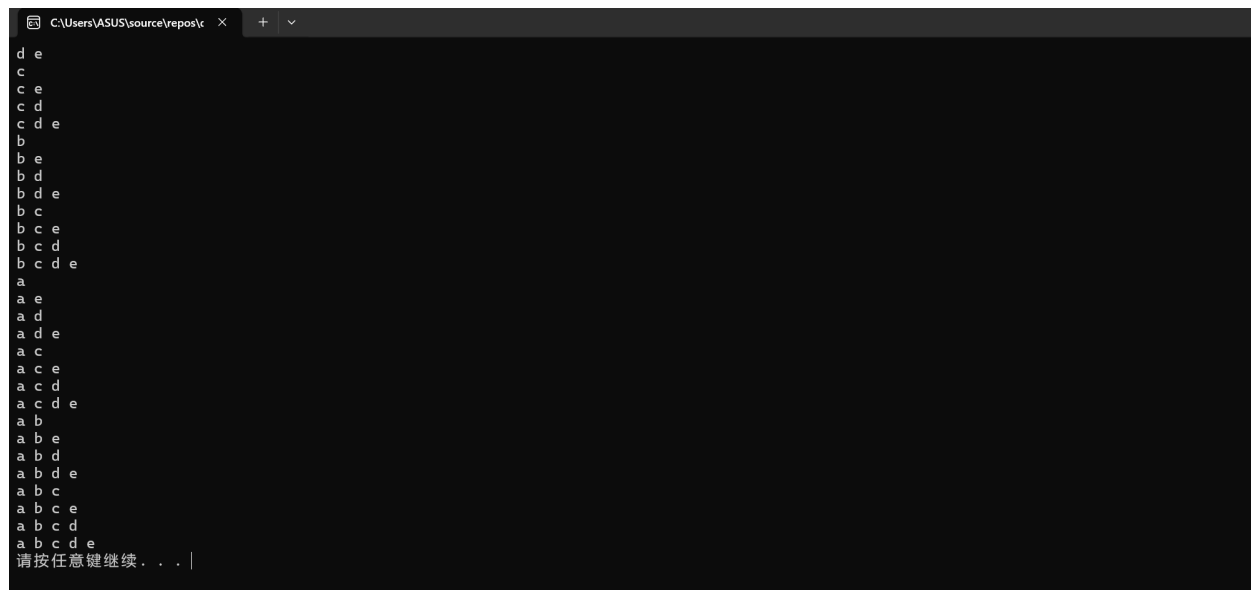


图 2.3: 测试用例 5 的输出结果



图 2.4: 测试用例-5 的输出结果